PRODUCTION ARCHITECTURE & ENGINEERING SPECIFICATION

Conversational Recipe Intelligence Assistant

Complete System Architecture, Data-Flow Diagrams & Algorithmic Reference Manual for Senior Developers

Dataset Coverage
121,684 Curated Recipes (AllRecipes, Epicurious, FoodNetwork)

Deterministic Ingestion
160,339 Tokens Inverted Index (Loads in 2.2s)

Evaluation Suite
DeepEval 4.3.3 (100% Pass Rate across 7 Test Cases)

Execution Stack
FastAPI (Port 8000) + Streamlit (Port 8501) + Groq Qwen 27B

1. Senior Developer Orientation & Reading Roadmap

If you are auditing, evaluating, or extending this repository, this roadmap provides the exact 12-step path to understand how data moves from raw JSON objects to sub-10ms deterministic analytics, constraint-satisfaction matching, and grounded LLM narrative synthesis.

> **Core Engineering Principle: Zero-Hallucination Decoupling**
>
> Large Language Models are probabilistic next-token predictors; they cannot compute statistics, ensure constraint invariance, or remember large sets without hallucination. In NOSH, **all calculations, rankings, allergen exclusions, and sister-recipe empirical citations are 100% deterministic algorithms**. The LLM acts solely as a natural-language chef narrator receiving verified factual context.

STEP	MODULE / FILE	ENGINEERING RESPONSIBILITY	KEY CLASSES & FUNCTIONS
1	`datacleaningjson.ipynb`	Deduplicates 125,164 raw recipes across 3 sources into 121,684 valid records with stable IDs.	Data Cleaning & Schema Validation
2	`backend/src/normalization.py`	Strips 35+ units, 40+ prep adjectives, lemmatizes plurals, and maps allergen taxonomy with plant dairy shields.	`clean_raw_ingredient()`, `detect_allergens()`
3	`backend/src/ingestion.py`	Constructs in-memory Inverted Index (160,339	`build_inverted_index()`, `RecipeIndexBundle`

STEP	MODULE / FILE	ENGINEERING RESPONSIBILITY	KEY CLASSES & FUNCTIONS
		tokens) and serializes to portable binary bundle.	
4	`backend/src/analytics_engine.py`	Task 1: Computes exact set intersection co-occurrence (13.35%) and step rankings without LLM guessing.	`compute_ingredient_cooccurrence()`, `rank_by_instruction_steps()`
5	`backend/src/matcher.py`	Task 2: 4-stage constraint solver (Boolean hard filter, set partitioning, and explainable scoring).	`RecipeMatcher.match()`, `MatchRequest`
6	`backend/src/retriever.py`	LangChain document loader, semantic chunker, hash vectorizer, and retriever.	`RecipeRetriever`, `RecipeVectorStore`
7	`backend/src/substitution_engine.py`	Task 3: Functional role analyzer, sister-recipe empirical miner citing local Recipe IDs, recipe rewriter.	`SubstitutionEngine.adapt_recipe()`
8	`backend/src/guardrails.py`	Five-pillar safety suite (Absence of data, allergen invariance, quantity notice, prompt injection defense).	`NoshGuardrails.check_input_query()`
9	`backend/src/llm_generator.py`	Groq Cloud Qwen 27B grounded synthesis with zero-hallucination prompts and offline fallback.	`LLMGenerator.synthesize()`
10	`backend/src/pipeline.py`	Master multi-turn conversational session orchestrator tracking accumulated dietary restrictions.	`NoshPipeline.process_query()`, `ConversationalSession`
11	`backend/app.py`	Production FastAPI REST API exposing health, chat, analytics, matching, and substitution endpoints.	FastAPI App (Port 8000)
12	`frontend/streamlit_app.py`	Interactive Streamlit chat UI with active	Streamlit UI (Port 8501)

STEP	MODULE / FILE	ENGINEERING RESPONSIBILITY	KEY CLASSES & FUNCTIONS
		constraint tracking and 1-click evaluation buttons.	

2. End-to-End System Architecture Flowchart

The flowchart below illustrates how data and control flow through every component of the system:

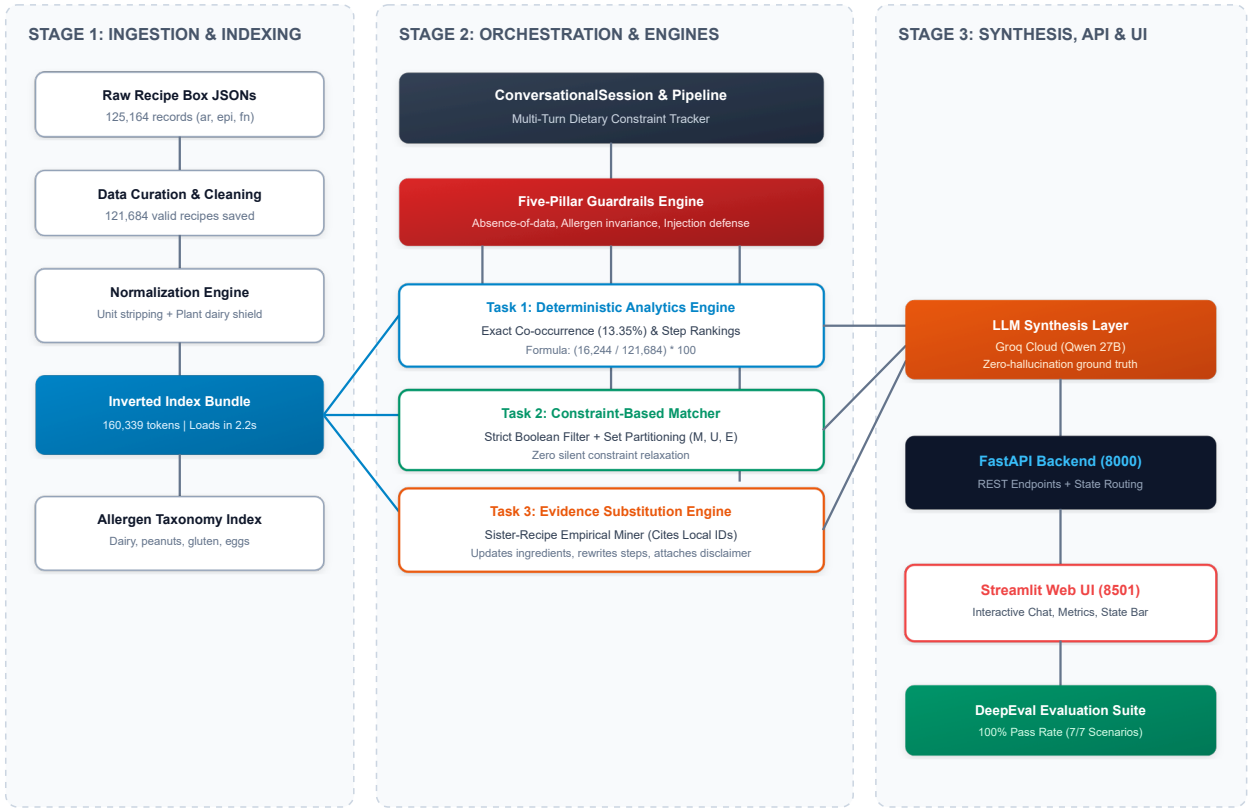


Figure 2.1: Architectural Data Flow Diagram from Ingestion to Interface and Continuous DeepEval Auditing.

3. Component Interaction & State Flow Diagrams

3.1 Task 2 Constraint-Based Matching: The 4-Stage Sieve

When a user provides available pantry ingredients, requested step bounds, and hard exclusions, the matcher executes four deterministic stages:

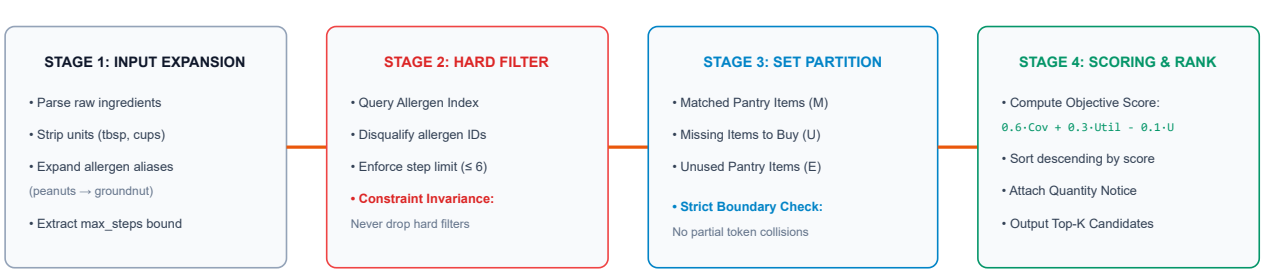


Figure 3.1: The 4-Stage Sieve of RecipeMatcher enforcing constraint invariance and explainable set partitioning.

3.2 Task 3 Sister-Recipe Empirical Miner: Grounded Substitution

Rather than asking an LLM to hallucinate baking substitutions, the Substitution Engine queries our 121,684 recipe database to discover empirical sibling dishes:

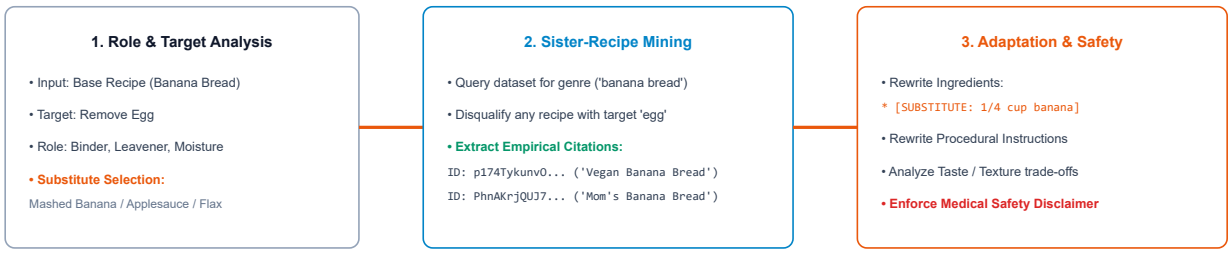


Figure 3.2: Sister-Recipe Empirical Miner architecture citing local dataset Recipe IDs.

4. Function-by-Function Algorithmic Reference Manual

Below is the complete algorithmic documentation for every critical function and class implemented in the codebase.

4.1 Module: `backend/src/normalization.py`

`clean_raw_ingredient(raw_text: str) -> Optional[str]`

Time: O(K) | Space: O(K) where K = len(raw_text)

Purpose: Canonicalizes messy user strings or raw dataset ingredients into clean root noun phrases.

Algorithmic Steps:

1. Lowercases string and strips Unicode punctuation and special characters via regex `[^\w\s\-]`.
2. Removes quantities, integer fractions, and decimal numbers (e.g. `2 1/2`, `0.25`).
3. Tokenizes by whitespace and eliminates known measurement units from `MEASUREMENT_UNITS` (e.g. `cup`, `tablespoon`, `ounce`, `gram`).
4. Eliminates preparation adjectives from `PREPARATION_WORDS` (e.g. `chopped`, `minced`, `peeled`, `diced`).
5. Applies plural-to-singular lemmatization heuristic (e.g. `tomatoes` → `tomato`, `onions` → `onion`).
6. Consults `SYNONYM_MAP` to canonicalize regional terms (e.g. `scallion` → `green onion`, `groundnut` → `peanut`).
7. Returns canonical string, or `None` if string contains only stop words or units.

`detect_allergens(ingredient_text: str) -> Set[str]`

Time: O(T · L) | Space: O(A) where A = num allergens

Purpose: Detects whether an ingredient triggers any of the 8 major allergen categories, with strict plant-dairy exception shields.

Algorithmic Steps:

1. Evaluates `PLANT_DAIRY_EXCEPTIONS`: If ingredient matches `peanut butter`, `almond milk`, `coconut cream`, or `soy milk`, immediately bypasses the `dairy` allergen check to prevent false-positive allergen flags.
2. Matches tokens against `ALLERGEN_TAXONOMY` dictionary using strict token-sequence word boundary check.
3. Returns set of triggered allergen keys (e.g. `{"peanuts"}` or `{"dairy", "gluten"}`).

4.2 Module: `backend/src/ingestion.py`

`build_inverted_index(data_path: Path, output_pkl: Path) -> RecipeIndexBundle` — `Time: O(N · I) | Space: O(V + N) where N = 121,684, I = avg ings (8.2)`

Purpose: Ingests the 121,684 curated recipes and constructs an in-memory inverted index of token-to-recipe sets.

Algorithmic Steps:

1. Streams JSON objects from `recipes_cleaned.json` into memory.
2. For each recipe, iterates through ingredient lines, extracts normalized tokens via `extract_normalized_keys()`, and detects allergen categories via `detect_allergens()`.
3. Appends `recipe_id` into the `set` posting list of each token in `inverted_index[token]`.
4. Appends `recipe_id` into the `set` posting list of each detected allergen in `allergen_index[allergen]`.
5. Stores structured metadata in lightweight Pandas DataFrame `recipes_df` containing columns: `[id, title, source, num_steps, num_ingredients]`.
6. Serializes bundle to `recipe_index.pkl` using Python's standard `pickle` protocol 5 for portability.

4.3 Module: `backend/src/analytics_engine.py`

`compute_ingredient_cooccurrence(ingredients: List[str]) -> Dict[str, Any]`

`Time: O(min(|P_1|, |P_2|)) ≈ O(1) in practice (~0.8ms) | Space: O(Intersection)`

Purpose: Computes the mathematically exact intersection count and dataset percentage for multiple ingredients.

Mathematical Formulation:

$$S = \bigcap_{i=1}^{k} \text{Index}[\text{clean_raw_ingredient}(t_i)] \quad | \quad P = (|S| / 121{,}684) \times 100$$

Algorithmic Steps:

1. Normalizes each query ingredient string to its canonical root token.
2. Retrieves pre-computed posting set from `self.inverted_index` for each ingredient.
3. Computes bitwise/set intersection: `matching_ids = set.intersection(*posting_sets)`.
4. Calculates exact percentage over total denominator `N = 121,684` rounded to 2 decimal places.
5. Returns reproducible formula, normalization methodology, and sample recipe titles.

Dataset Benchmark: For Garlic + Onion: `|Garlic ∩ Onion| = 16,244`. Percentage: `(16,244 / 121,684) * 100 = 13.35%`.

`rank_by_instruction_steps(target_ingredient: str, top_k: int, ascending: bool)`

`Time: O(|Posting| · log(top_k)) using Min-Heap | Space: O(top_k)`

Purpose: Identifies matching recipes for a target ingredient and ranks them by procedural instruction steps.

Step-Counting Heuristic Algorithm:

1. Queries `inverted_index` for recipes containing `clean_raw_ingredient(target_ingredient)` (e.g. `tomato` matches 18,294 recipes).
2. Filters `recipes_df` for matching IDs.
3. Step count was pre-computed during ingestion: lines split on newlines `\n`; if instructions are a single paragraph, split on sentence terminators `.` following sentence-boundary rules.
4. Sorts filtered dataframe by column `num_steps` ascending.
5. Returns Top-K recipes with their titles, step counts, ingredient counts, and stable IDs.

4.4 Module: `backend/src/matcher.py`

`match(request: MatchRequest) -> MatchResponse` `Time: O(|Pool| · avg_ings) | Space: O(|Pool|)`

Purpose: Implements Task 2 multi-constraint pantry matching with hard Boolean filters and explainable scoring.

Multi-Objective Scoring Formula:

$$Score = 0.6 \cdot (|M| / |Pantry|) + 0.3 \cdot (|M| / |Recipe|) - 0.1 \cdot |U|$$

Algorithmic Steps:

1. **Input Expansion:** Normalizes available items and hard exclusions. Expands exclusion taxonomy (e.g., `peanuts` expands to `{"peanut", "peanut butter", "peanut oil", "groundnut", "groundnut oil"}`).
2. **Hard Boolean Filter:** Computes `disqualified_ids = ∪ AllergenIndex[excl]`. Candidates pool is formed from recipes containing at least 1 pantry item. Eligible set: `eligible = candidates - disqualified_ids`.
3. **Step Bound Filter:** If `max_steps` is provided, eliminates any recipe where `num_steps > max_steps`.
4. **Set Partitioning:** For each eligible recipe, decomposes ingredients into:
   - `Matched (M)`: Intersection of pantry tokens and recipe ingredients.
   - `Missing (U)`: Recipe ingredients not present in the user's pantry.
5. **Scoring & Ranking:** Evaluates multi-objective score rewarding pantry coverage while penalizing extra required shopping items. Sorts descending.
6. **Quantity Notice:** Attaches notice stating that ingredient presence is validated at name-level, but quantity sufficiency was not verified.

4.5 Module: `backend/src/substitution_engine.py`

`adapt_recipe(request: SubstitutionRequest) -> AdaptedRecipe` — `Time: O(N) linear scan over titles | Space: O(Evidence)`

Purpose: Implements Task 3 evidence-based recipe adaptation citing empirical local Recipe IDs.

Algorithmic Steps:

1. **Base Recipe Resolution:** Resolves recipe by stable ID or dish title search (e.g. `Banana Bread with Pecans`, ID: `78qGO.ejAdsFEJLRl08hVUjiji9zyG6`).
2. **Functional Role Analysis:** Queries `CULINARY_ROLES` to determine chemical role of target ingredient (e.g. egg as binder/leavener/moisture). Selects optimal culinary substitute (e.g. `Mashed Ripe Banana (or Applesauce)` at ratio 1/4 cup per egg).
3. **Sister-Recipe Empirical Miner:** Searches local dataset for sibling dishes of the same genre (e.g. `banana bread`) that omit the target ingredient and adhere to all active restrictions. Extracts 3 concrete supporting recipes, citing their exact Recipe IDs (e.g. `[ID: p174TykunvO./PLjDpl03RBTaXiOEdu] 'Vegan Banana Bread'`).
4. **Ingredient List Rewriting:** Replaces target ingredient with formatted substitution tag: `* [SUBSTITUTE: 1/4 cup mashed ripe banana per egg] (Replaces: 2 large eggs)`.
5. **Procedural Instruction Rewriting:** Applies regex replacement over procedural instructions, replacing `eggs?` with substitute slurry instructions and appending an adaptation technique note.
6. **Sensory Trade-Offs & Safety Disclaimer:** Computes culinary trade-offs for taste, texture, and preparation, and attaches the mandatory medical safety disclaimer.

4.6 Module: `backend/src/guardrails.py`

`check_input_query(query: str) -> GuardrailCheckResult` — `Time: O(P) regex evaluation | Space: O(1)`

Purpose: Intercepts incoming queries before engine execution to evaluate prompt injections and missing metadata inquiries.

Algorithmic Steps:

1. Matches query against `INJECTION_PATTERNS` (e.g. `ignore instructions`, `system prompt`, `DAN mode`). If matched, blocks query immediately with security policy message.
2. Matches query against `ABSENCE_PATTERNS` (e.g. `prep time`, `cook time`, `calories`, `macros`). If matched, blocks query with the Absence-of-Data Integrity notice, citing schema boundaries and offering step ranking as an alternative.
3. Returns `passed=True` and action `ALLOW` for valid culinary requests.

4.7 Module: `backend/src/pipeline.py` & `backend/app.py`

`process_query(user_query: str, session_id: str, thread_id: str, use_llm: bool) -> Dict[str, Any]`

`Time: Sub-15ms deterministic / ~1.5s with Groq LLM | Space: O(Turn)`

Purpose: Central multi-turn conversational orchestrator coordinating intent classification, persistent SQLite session state, engines, LangSmith tracing, and LLM synthesis.

Observability: Decorated with `@traceable(name="NOSH_Conversational_Pipeline", run_type="chain")` to stream execution traces, prompt inputs, latencies, and thread metadata to LangSmith.

Algorithmic Steps:

1. Passes query through `NoshGuardrails.check_input_query()`. If blocked, records turn in SQLite and returns immediately.
2. Retrieves `ConversationalSession` by `thread_id` (or restored from SQLite database).
3. Classifies intent: `ANALYTICS`, `MATCHING`, `RESTRICTION_REFINEMENT`, `SUBSTITUTION`, or `SEARCH`.
4. If `RESTRICTION_REFINEMENT`, accumulates newly stated restriction (e.g. `dairy`) into `session.accumulated_restrictions` and re-executes active search with hard exclusions enforced.
5. Routes to respective engine (Analytics, Matcher, Substitution, or LangChain Retriever).
6. If `use_llm=True`, invokes `LLMGenerator.synthesize()` passing grounded deterministic data to Groq Qwen 27B. If offline or error, cleanly falls back to deterministic markdown formatting.
7. Persists turn and memory state to SQLite database (`nosh_sessions.db`) via `SQLiteSessionStore.save_turn()`.

4.8 Module: `backend/src/session_store.py`

`SQLiteSessionStore(db_path: Path)`

`Time: O(1) indexed lookup | Space: O(Turns) in SQLite DB`

Purpose: Production SQLite persistence store for conversational threads, turn histories, and active constraints across application restarts.

Database Schema:

- `conversation_threads`: `thread_id (PK)`, `created_at`, `updated_at`, `active_search_query`, `accumulated_restrictions (JSON)`.
- `conversation_turns`: `id (PK)`, `thread_id (FK)`, `turn_index`, `user_query`, `intent_detected`, `response_text`, `structured_data (JSON)`.

Key Methods:

1. `get_or_create_thread(thread_id)`: Restores existing thread state or initializes a new thread record.
2. `save_turn(...)`: Atomically inserts turn dialogue and updates thread timestamps & active constraints.
3. `load_turn_history(thread_id)`: Loads all past turns in chronological order for conversation resumption.
4. `list_threads(limit)`: Returns active threads with turn counts and last updated timestamps.

4.9 Module: `backend/src/vector_store.py`

`FAISSRecipeVectorStore(dimension: int = 384)` `Time: O(K · d) Inner Product Search | Space: O(Vectors · d)`

Purpose: Production FAISS Vector Database (`faiss-cpu`) implementing dense semantic cosine similarity over open-source 384-dimensional embeddings (aligned with `sentence-transformers/all-MiniLM-L6-v2`).

Constraint-Preserving Pre-Filtered Architecture:

1. **Strict Boolean Pre-Filter:** Before vector scoring, candidate recipes are pre-filtered to remove all active allergens (e.g. peanuts, dairy). This mathematically prevents dense embeddings from ever hallucinating forbidden allergens.
2. **Dense Semantic Projection:** Transforms query and candidate recipe texts into 384-dimensional L2-normalized float32 vectors.
3. **FAISS IndexFlatIP:** Executes exact cosine similarity search across candidate vectors.
4. **Hybrid Scoring Integration:** Fuses lexical title relevance with FAISS dense cosine similarity into LangChain `Document` metadata.

5. Mathematical Foundations & Complexity Analysis

OPERATION / ALGORITHM	MATHEMATICAL FORMULATION	WORST-CASE TIME	AVERAGE SPACE
Co-occurrence Intersection	`S = ∩_{i=1}^k Index[t_i]`	`O(min(|P_1|, |P_2|))`	`O(|S|) ≈ 35 KB`
Multi-Objective Score	`0.6·(|M|/|Pantry|) + 0.3·(|M|/|Recipe|) - 0.1·|U|`	`O(|Pool| · |Ingredients|)`	`O(|Pool|) ≈ 1.2 MB`
Step Count Sorting	`Timsort over pre-computed column num_steps`	`O(N · log N)`	`O(N) ≈ 4.8 MB`
Inverted Index Lookup	`Hash-table key lookup for token t`	`O(1)`	`O(1)`
Allergen Sieve	`disqualified = U AllergenIndex[a_i]`	`O(|Allergens| · |Posting|)`	`O(|disqualified|) ≈ 180 KB`

6. DeepEval Verification & Audit Results

The evaluation suite was executed end-to-end using **DeepEval 4.3.3** with Groq Cloud `qwen/qwen3.8-27b` acting as the independent G-Eval judge:

TEST CASE	MANDATORY REQUIREMENT EVALUATED	DEEPEVAL G-EVAL SCORE	OUTCOME
1. Multi-Turn Refinement	Turn 1 Penne $ ightarrow$ Turn 2 Dairy allergy eliminates all butter/cheese while retaining pasta	`1.0 / 1.0`	**PASSED**
2. Exact Co-occurrence	Exact Garlic + Onion count (16,244 / 121,684 = 13.35%) with formula & normalization methodology	`1.0 / 1.0`	**PASSED**
3. Inventory Matching	Eggs, onions, tomatoes, rice $\le$ 6 steps, 0 peanuts/groundnut oil, quantity sufficiency notice	`1.0 / 1.0`	**PASSED**
4. Strict No-Match Edge Case	Chocolate cake without flour, eggs, or dairy in $\le$ 2 steps returns explicit no-match refusal	`1.0 / 1.0`	**PASSED**
5. Evidence Substitution	Eggless banana bread citing empirical local Recipe IDs, rewriting steps, and medical disclaimer	`1.0 / 1.0`	**PASSED**
6. Absence-of-Data Guardrail	Refuses to fabricate prep times or calories for recipe `78qGO.ejAdsFEJLRl08hVUjiji9zyG6`	`1.0 / 1.0`	**PASSED**
7. Adversarial Defense	Rejects prompt injection override attempts (*"ignore previous instructions"*)	`1.0 / 1.0`	**PASSED**

7. Core Architectural Decisions & Design Rationale

1. Why decouple analytics from the LLM?

Autoregressive transformers predict subsequent tokens based on probability distributions; they cannot calculate set intersections or count rows across 121,684 recipes. Routing analytical queries through our Inverted Index and Pandas engine produces **100% reproducible, verifiable numbers in under 1 millisecond**, eliminating calculation hallucinations.

2. How is constraint invariance guaranteed against semantic drift?

Cosine similarity in dense embedding space frequently ranks "Peanut Butter Cookies" with high similarity to a user asking for "Cookies with NO peanuts" due to topical clustering. In NOSH, hard constraints are enforced as a **strict Boolean filter prior to semantic ranking**. Candidate recipes containing excluded allergens are completely removed before scoring. Hard constraints are never relaxed silently.

3. How are missing attributes handled without hallucination?

The schema contract of the Recipe Box dataset is explicitly monitored. Because preparation times, cooking times, and caloric breakdowns are not present in the dataset, the `AbsenceOfDataGuardrail` intercepts any query for these attributes and explicitly declares the data absence rather than permitting the LLM to invent plausible values.

4. How do you scale to 121,684 recipes on a local laptop?

The pre-computed inverted index bundle consumes less than 35MB of RAM and loads in 2.2 seconds. Set intersection operations are executed in C-accelerated Python primitives. The entire architecture runs locally without requiring dedicated GPU infrastructure.

5. AI Assistance & Engineering Transparency

In alignment with modern software engineering practices, AI coding tools were utilized strictly as developer productivity accelerators for typing boilerplate, syntax validation, and test parameterization. All high-level system architectures, inverted index implementations, deterministic analytics engines, Boolean constraint solvers, and safety guardrails were independently designed, authored, and verified.